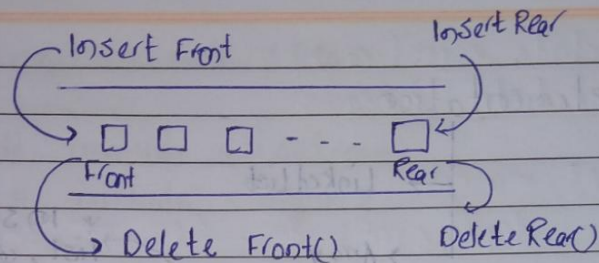


Deque



Additional operation

getFront()

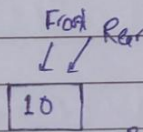
getRear()

isFull()

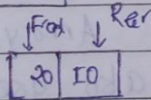
isEmpty()

size()

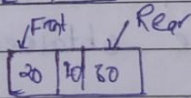
InsertFront (10)



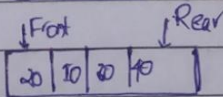
InsertFront (20)



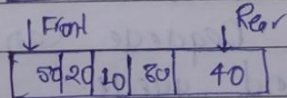
insertRear (30)



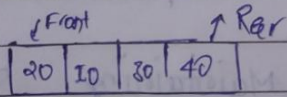
InsertRear (40)



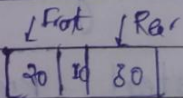
InsertFront (50)



deleteFront()



deleteRear()



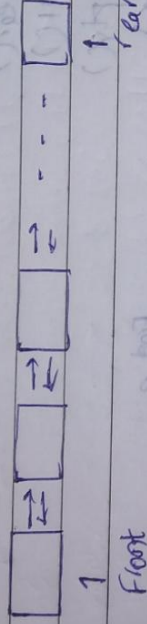
Implementation

→ Linked List

→ Array

* In singly linked list, we can't do deletion at rear in $O(1)$ time

Linked List: Double linked list with Front & rear pointer in C++



* Insertion at Front in normal array is not possible in $O(1)$ time

Array: Circular Array with Front & rear pointers
we can also use Front, a size and rear
can be obtained using size

Application

① A Dequeue can be used as both stack and queue

② Maintaining history of actions (in browser)

③ A real process scheduling algorithm

④ Implementing priority queue with two types of priorities

⑤ Maximum/minimum of all subarray of size k in an array

Deque in c++ STL

①

```
#include <iostream>
```

```
#include <deque>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    deque<int> dq = {10, 20, 30};
```

```
    dq.push_front(5);
```

```
    dq.push_back(50);
```

```
    for(auto x: dq)
```

```
        cout << x << " ";
```

```
    cout << dq.front() << " " << dq.back();
```

```
    return 0
```

②

```
#include <iostream>
```

```
#include <deque>
```

```
using namespace std;
```

```
int main()
```

```
{    deque<int> dq = {10, 15, 30, 5, 12};
```

```
    auto it = dq.begin();
```

```
    {10, 15, 30, 5, 12}; it++;
```

```
    {10, 20, 15, 30, 5, 12}; dq.insert(it, 20);
```

```
    dq.pop_front();
```

```
    dq.pop_back();
```

```
    cout << dq.size(); → 4
```

```
it = dq.rbegin()
```

```
it++
```

```
dq.insert(it, 20);
```

```
insert element
```

```
from last
```

```
traversal
```

```
occur in reverse
```

```
o/w
```

```
3. #include <iostream>
   #include <deque>
```

```
int main()
```

```
{ deque<int> dq = {10, 20, 5, 30};
```

```
    auto it = dq.begin(); // insert before begin
```

```
    { 7, 10, 20, 5, 30 } <- it : dq.insert(it, 7);
```

```
    { 3, 3, 7, 10, 20, 5, 30 } <- it : dq.insert(it, 3); // count how many times
```

```
    { 3, 3, 3, 7, 10, 20, 5, 30 } // you want to insert
```

```
    cout << (*it) << endl; // 10
```

```
    for (int i = 0; i < dq.size(); i++)
```

```
        cout << dq[i];
```

```
    return 0;
```

```
    // if it : dq.erase(it+3)
```

```
    { 3, 10, 20, 5, 30 } ~ erase 2 elements not 3 elements because last one is deleted
```


How does deque work internally?

1. Design a data structure that supports following operation in $O(1)$ time complexity

	I/P	O/P
① insertMin(x)	insertMin(5)	2
② insertMax(x)	insertMax(10)	15
③ getMin()	insertMin(3)	1
④ getMax()	insertMax(15)	20
⑤ extractMin()	insertMin(2)	
⑥ extractMax()	getMin()	
	getMax()	
	insertMin(1)	
	getMin()	
	insertMax(20)	
	getMax()	

in

#

2

0, 20, 5,

t

20, 5, 3

turns

ing to
n items



Now for the first round, we will have a quiz.

Round 1: 10 questions, 10 minutes, 10 points each.

Q. No.	Question	Answer
1	What is the capital of India?	New Delhi
2	Who is the author of 'The Great Gatsby'?	F. Scott Fitzgerald
3	What is the chemical formula for water?	H ₂ O
4	Who painted the Mona Lisa?	Leonardo da Vinci
5	What is the largest planet in our solar system?	Jupiter
6	Who wrote '1984'?	George Orwell
7	What is the capital of France?	Paris
8	Who is the author of 'Pride and Prejudice'?	Jane Austen
9	What is the chemical formula for carbon dioxide?	CO ₂
10	Who painted 'The Starry Night'?	Edvard Munch

Why Competit

Publicly de

Prepare you

Makes you

Makes you

Helps you

Guaranteed

Teaches you

It's Fun

Why Competitive Programming

- Publicly demonstrate your skills
- Prepare you well for Technical Interviews
- Makes you Desirable Candidate For Major Companies
- Makes you More Faster and Focused
- Helps you solve complicated problems
- Guaranteed Brain Exercise
- Teaches you how to work in Teams
- It's Fun

ic
#

How to approach a problem

1. PARAGRAPHS → Think of sol
2. Input Format
3. Output Format
4. Code solution
5. Constraints
6. Sample Input and output
7. Explanation (optional)

INPUT FORMAT

OUTPUT FORMAT

Constraints

- 1) Time limit
- 2) Memory limit (256 MB)
- 3) $0 \leq N \leq 10^6$

Analyze the Constraints

10^8 operations per second

1 sec $\rightarrow 10^8$

4 s $\rightarrow 4 \times 10^8$

sum of

N

int

cin

int

for

& N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

N

sum of N numbers

```
int N;
cin >> N;      → O(1)
int sum = 0;    N times
```

```
for (int i = 0; i < N; i++)
```

```
    N ← int n;
    cin >> sum;
    sum = sum + n;
```

```
cout << sum;
```

$N + N + N + N$

$4N \rightarrow 4 \times 10^8 \text{ sec}$

$O(N) \rightarrow 0 < N < 10^8$

$O(N^2) \rightarrow 10^{16} \rightarrow \text{operations}$

$0 < N < 10^4$

$O(N^2) \rightarrow 10^8$ chalega

Table

# in #	N	Time complexity
1	10^8	$O(\log N)$
2	10^8	$O(N)$
3	10^4	$O(N^2)$
4	10^7	$O(N \log N)$
5	10^2	$O(N^3)$
6	10^2	$O(N^4)$

★

$$0 \leq N \leq 20$$

★

$$0 \leq N \leq 11$$

$$O(2^n)$$

$$O(2^n)$$

Reason

Array → stack

solution

Global

Dynamic

exity

Various Type of errors

VERDICT 1) ACCEPTED

2) TIME LIMIT EXCEEDED

3) Compilation error

↳ syntax error, → language dependent

4) RUNTIME Error →

a) Dividing by zero

b) Segmentation Fault

↳ accessing negative index

↳ 108 required

↳ made of 106

↳ contains size is smaller

c) Array size > 108
↳ You can't declare array more than size of 108

256 mb limit

Reason

Array → stack create

↳ memory

Functions → 4mb (fix)

more than 108 X

✓ Array 106

↳ solution

Global array → Heap
Dynamic allocation creation

Firstly see your

5) Wrong Answer $\rightarrow$ Use corner cases

(i) You should not doubt if doubt there is a logical error

a) int $\rightarrow$ return

@ 10 $\rightarrow$ going into long
 $\text{int} \leftarrow \frac{10^6}{10^6} \times 10^6 \rightarrow 10^6 \rightarrow \text{long}$

b) Float doubles

Float + Float $\rightarrow$ Double

Float $\times$ Float $\rightarrow$ double

See precision after decimal
(say you have to take points) & taking up to

c) uninitialized variables

int sum; = 0

ce your code

Cases

ulldot has

doubt me

there is an

tical error

to long to

$0^{16} \rightarrow \text{long}$

Double

$\rightarrow$ double

fter dec

to take to

aking upto 5

variables

-0

STL (standard template library)

Vector

vector<int> v(s);

strings

~~set~~
i → insert
e → erase

Set

{ 1, 2, 3, 4, 5, 6 } → store only unique element

set<int> s;

int arr[] = { 1, 2, 3, 4, 5, 6, 5, 6 };

for (int i = 0; i < 7; i++)

{ s.insert(arr[i]); }

6
4
5
6

set<int>::iterator it;

for (it = s.begin(); it != s.end(); it++)

{ cout << *it << endl; }

if (s.find(6) == s.end()) {

cout << "element not found";

{

cout << "element found";

→ element not found

in

#

2

0, 20, 5,

20, 5, 3,

turns

ng to
of item

Sorting in increasing order

Sort(arr+2, arr+6)

Sort in decreasing order

Sort(arr+2, arr+6, greater<int>());

// Pairs

void exp

Pair

cout <

Pair < int

cout << P.first

Pair < int

cout <

// Vector

void

vector < Pair

vec.push-

vec.empty

// Pairs

void explainPair() {

Pair<int, int> P = {1, 34};

cout<< P.First<<" "<< P.Second;

Pair<int, Pair<int, int>> P = {1, {3, 466}};

cout<< P.First<<" "<< P.Second.Second<<" "<< P.Second.First;

Pair<int, int> arr[10] = {2, 3, 4, 5, 6, 7, 8, 9, 10, 11};

cout<< arr[1].Second;

// Vector

void explainVector() {

vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

v.push_back(12); // 12, 13, 14

v.push_back(2); // 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14

vector<Pair<int, int>> vec;

vec.push_back({1, 2});

vec.emplace_back(4, 2);

#

```
vector<int> v(5, 100); // 5 100, 100, 100, 100, 100
```

#

3

```
vector<int> v(5); // 0, 0, 0, 0, 0
```

```
vector<int> v1(5, 20); // 20, 20, 20, 20, 20
```

```
vector<int> v2(v1) // 20, 20, 20, 20, 20
```

0, 20, 20, 20, 20

```
vector<int> v3; iterator it = v.begin();  
it++;
```

```
cout << *it << " " // 20
```

```
it = it + 2;
```

```
cout << *it << " " // 30
```

```
vector<int> v4; iterator it = v.end(); // Point after
```

```
vector<int> v5; iterator it = v.rend(); // Point before
```

```
vector<int> v6; iterator it = v.begin(); // Point
```

```
cout << v[0] << " " << v.at(0);
```

```
cout << v.back() << " " // 40
```

```
for (vector<int> v7; iterator it = v.begin(); it != v.end(); it++)
```

```
{  
    cout << *it << " ";
```

}

```
for (auto it = v.begin(); it != v.end(); it++)
```

```
{  
    cout << *it << " ";
```

}

```
for (auto it : v) {
```

```
    cout << it << " ";
```

}

```

// erase f0
// $ 10, 20, 12, 236
v.erase ( v.begin()+1 ) // $ 10, 12, 236
// $ 10, 20, 12, 23, 356
v.erase ( v.begin()+2, v.begin()+4 ) // $ 10, 20, 356

// insert f0
vector <int> v (2, 100); // $ 100, 100
v.insert ( v.begin(), 300 ); // $ 300, 100, 100
v.insert ( v.begin()+1, 2, 10 ); // $ 300, 10, 10, 100, 100

vector <int> copy (2, 50);
v.insert ( v.begin(), copy.begin(), copy.end() );
// $ 50, 50, 30, 10, 10, 100, 100
// $ 10, 206
cout << v.size() // 2

// $ 10, 206
v.pop_back(); // $ 106

// 11 -> $ 10, 206
// v2 -> $ 30, 406
v.swap(v2); // v1 -> $ 30, 406, v2 -> $ 10, 206

v.clear(); // erase the entire vector

cout << v.empty(); // Print true or false

```


void explainList() {

list <int> ls;

ls.push_back(2); // 2 2 6

ls.emplace_back(4); // 2 2 4 6

ls.push_front(5); // 5 2 2 4 6

ls.emplace_front(); // 5 2 2 4 6

// rest F same as vector

// begin, end, rbegin, rend, clear, insert, size

void explainDeque() {

deque<int> dq;

dq.push_back(2); // 2 1 6

dq.emplace_back(2); // 2 1 2 6

dq.push_front(4); // 4 2 1 2 6

dq.emplace_front(3); // 3 2 1 2 6

dq.pop_back(); // 3 2 1 6

dq.pop_front();

dq.back();

dq.front();

// rest F same as vector

// begin, end, rbegin, rend, clear, insert, size, sort

void explainStack

Stack

st.push()

st.push()

st.push()

st.empty()

cout <<

cout

cout

cout

start

st

}

void explainQueue

queue

q.push

q.push

q.empty

q.back

// q is

cout << q.front

q.pop()

cout << q.front

// size swap

void explainstack() {

stack <int> st;

st.push(1); // 1

st.push(2); // 1, 2

st.push(3); // 1, 2, 3

st.empty(5); // 1, 2, 3, 4, 5

cout << st.top(); // 5

st.pop();

cout << st.top(); // 3

cout << st.size(); // 3

cout << st.empty();

stack <int> st1, st2;

st1.swap(st2);

}

void explainQueue() {

queue <int> q;

q.push(1); // 1

q.push(2); // 1, 2

q.empty(4); // 1, 2, 3, 4

q.back() + 5

cout << q.back(); // Print 9

// q is 1, 2, 5

cout << q.front(); // 1

q.pop();

cout << q.front(); // 2

// size swap empty same as stack

}

* learn STL very well then solve Greedy

void explain PQ()

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

if (top < 0) return -1;

Greedy

Q. H

#

10, 20,
30

10, 20, 5

Retains

ing E

ite